

Running the Micro Eazy ecosystem

Four repositories. Seven systems. One identity. This is the manual for the gap between those three numbers — every command that starts something, every setting that puts it on the internet, and the handful of places where getting it wrong is expensive rather than merely annoying.

COMPILED

24 August 2026

META-REPO

emmanuel-arch/microeazy

SOURCE OF TRUTH

ecosystem/registry.json

CONTENTS

01	Four repositories, seven systems	Why those numbers differ, and what it means for every command below
02	Starting the estate locally	Doctor, bootstrap, dev — and what to do when a port is taken
03	The command reference	Every npm script worth knowing, grouped by what you are trying to do
04	What changed today	The system switch, the seven doors — and the one migration that must run first
05	Vercel: four projects	Root directories, build commands, and the reason the Interchange 404s
06	Databases	Why the Interchange gets its own Supabase project and not a schema
07	DNS	Every record, in the order they should be added
08	Go-live, in order	The sequence, with the check that proves each step landed
09	Still open	What is not done, stated plainly

Every orchestration script reads ecosystem/registry.json and nothing hard-codes a path, a port or a command anywhere else. Where this document and that file disagree, the file is right — and this document is out of date.

Four repositories, seven systems

This is the single most confusing thing about the estate, and almost every mistake made against it traces back here. *Seven systems* is what a lender buys. *Four repositories* is what you deploy. Six of the seven systems live inside one repository.

The seven systems — what a lender sees

SYSTEM	WHAT IT IS FOR	ROUTE	LIVES IN
 Lending Console	Originate, score, disburse, collect	/console	connected-suite
 Customer Portal	The borrower's own door	/microeazy	connected-suite
 Analytics Studio	The whole book, drawn	/analytics	connected-suite
 PeopleHub HR	The roster behind the book	/people	connected-suite
 Ledgerly	The journal, read live	/books	connected-suite
 ConnectDesk	The collections floor	/desk	connected-suite
 The Interchange	Consent-gated exposure exchange	(own origin)	interchange

The four repositories — what you deploy

FOLDER	REPOSITORY	STACK	PORT	DEPLOYS AS
connected-suite	emmanuel-arch/BirgenAI_LMS	Next 16	3000	1 Vercel project, 6 systems
interchange	emmanuel-arch/Interchange	Next 16	3341	1 Vercel project
pwa	emmanuel-arch/micro-eazy-pwa	Vite + React	5173	1 Vercel project
pbx-auth	skegode/PBXAuthentication	.NET Worker	—	Self-hosted, no browser surface

THE FOLDER NAME IS NOT THE REPOSITORY NAME

The local folder `connected-suite` pushes to a repository still called `BirgenAI_LMS`, and its Vercel project has never seen the folder name — Vercel builds from the repository. Renaming the folder was safe for exactly that reason. Do not rename the *repository* to match: that would re-point a live project and re-map every environment variable by hand, for no product benefit.

Why one deployment serves six systems

Because they share the two surfaces where a multi-tenant bug becomes a cross-lender data leak: **auth and tenancy**. Six separate deployments would mean six copies of session handling, the daily OTP, tenant resolution, the vault and RBAC — and five chances for one of them to drift. A subdomain here is a *rewrite* (`src/proxy.ts`), not a second application.

The Interchange is the exception, and deliberately so. It is a separate *trust domain*, not merely a separate product: it brokers between competing lenders, and the architectural claim is that no member can read the Registry from their own systems. That claim is worth a second deployment and a second database. See 06.

Starting the estate

Everything runs from the meta-repo root, `C:\GIT\MICRO EAZY`. Four commands cover the normal day.

```
# From C:\GIT\MICRO EAZY — the meta-repo root

npm run doctor      # is this machine ready? checks submodules, deps,
                    # env files, free ports, required variables

npm run bootstrap   # npm install in every system, in sequence

npm run dev         # start every system at once, colour-tagged output
npm run dev interchange # ...or just one

npm run build       # production build of every system, sequentially
npm run start       # run those production builds
```

Keeping the pins in step

The meta-repo does not contain the systems; it *pins* them at a commit. That pin is the record of which combination constitutes a working estate — which was previously written down nowhere, and is precisely why a fresh machine could not reproduce one.

```
npm run pull        # pull the meta-repo and check out the pinned commits
npm run sync        # pull each submodule to the TIP of its branch instead

# After sync, the pins have moved. Record the new combination:
git add connected-suite interchange pwa
git commit -m "Bump the pointers"
```

A CHANGE SPANNING TWO SYSTEMS IS TWO PULL REQUESTS AND A POINTER BUMP

That is the cost of independent deployability, and it is worth naming rather than discovering. Push the system repository first, then bump the pin in the meta-repo. A pin bumped to a commit that was never pushed will check out on your machine and on nobody else's.

Ports, and what to do when one is taken

PORT	SYSTEM	CLAIMED IN
3000	Connected Suite	registry.json → portMap
3341	Interchange console	registry.json → portMap
5173	Micro Eazy PWA	registry.json → portMap
51214	Interchange PGLite (prisma dev)	registry.json → portMap

`npm run doctor` reports a busy port before `npm run dev` fails on it. Two systems both defaulting to 3000 is the single most common way a local ecosystem stops booting — claim the next free port in `portMap` *before* writing a new registry entry, not after.

The Interchange needs its database started first

It is the only system with a preDev step. `npm run dev` runs it automatically; if you are starting the console on its own, run it yourself.

```
# cd interchange/apps/interchange-console  
npx prisma dev -d -n interchange # detached; a no-op when already up  
npm run dev                      # :3341
```

The command reference

The Connected Suite alone carries eighty-six npm scripts. These are the ones worth knowing, grouped by what you are actually trying to do. Run `npm run` with no argument in any system to see the rest.

SUITE

PORTAL

ANALYTICS

PEOPLE

LEDGERLY

DESK

Connected Suite — `cd connected-suite`

Running it

COMMAND	WHAT IT DOES
<code>npm run dev</code>	Dev server on <code>:3000</code>
<code>npm run build</code>	<code>prisma generate</code> then <code>next build</code> — exactly what Vercel runs
<code>npm run start</code>	Serve the production build
<code>npm run lint</code>	ESLint

Database

COMMAND	WHAT IT DOES
<code>npm run db:push</code>	Push the Prisma schema to <code>DATABASE_URL</code> . Required before this release deploys — see 04
<code>npm run db:studio</code>	Prisma Studio, a browser over the data
<code>npm run db:seed</code>	Base seed
<code>npm run db:seed:demo</code>	The guided showcase org and its lifecycle
<code>npm run db:rls</code>	Apply row-level security policies
<code>npm run db:rls:verify</code>	Prove the policies actually bite
<code>npm run db:app-role</code>	Create the restricted application role
<code>npm run db:index-advisor</code>	Report indexes worth adding

Setting up a lender

COMMAND	WHAT IT DOES
<code>npm run platform:admin</code>	Create the BirgenAI platform administrator — the account that opens <code>/platform</code>
<code>npm run setup:launch</code>	Create the launch organisations
<code>npm run org:activate</code>	Flip an org to <code>ACTIVE</code> from the terminal
<code>npm run storage:init</code>	Create the storage buckets
<code>npm run vault:seed-micromart</code>	Seed Micromart's integration vault

Artwork

COMMAND	WHAT IT DOES
<code>npm run art:prompts</code>	Print each login plate's generation prompt and target path
<code>npm run art:optimize</code>	Report the PNG→WebP encode; writes nothing
<code>npm run art:optimize -- --write</code>	Actually encode. 8.02 MB → 230 kB
<code>npm run art:check</code>	New. Score each plate: left-third darkness, chroma, hue against its accent. Non-zero exit on a failure

Verification — the ones to run before a release

There are fifty of these. They are real assertions against the real database, not unit tests. These six cover the surfaces that break silently:

COMMAND	PROVES
<code>npm run test:access</code>	The access catalogue and per-person module denial
<code>npm run test:rbac</code>	Rights gate what they claim to gate
<code>npm run test:scope</code>	Tenant scoping — the cross-lender boundary
<code>npm run test:platform-admin</code>	The platform session cannot be forged
<code>npm run test:federation</code>	Cross-system session carry
<code>npm run test:live</code>	The live data reads behind the launcher tiles

EXCHANGE

The Interchange — `cd interchange/apps/interchange-console`

COMMAND	WHAT IT DOES
<code>npm run db:dev</code>	Start the local PGlite Postgres on :51214. Run this first
<code>npm run dev</code>	Console on :3341
<code>npm run build</code>	Production build
<code>npm run db:push</code>	Push the Registry schema
<code>npm run db:seed</code>	Seed members and reference data
<code>npm run keys:dev</code>	Generate the OPRF and member keys for local work
<code>npm run keys:session</code>	Generate INTERCHANGE_SESSION_SECRET
<code>npm run operator -- create</code>	Create an operator and their four-digit access code
<code>npm run operator:list</code>	List operators
<code>npm run verify:operator</code>	Prove a hand-written session cookie is refused
<code>npm run verify:exchange</code>	Prove the consent gate holds
<code>npm run verify:tamper</code>	Prove a tampered payload is rejected
<code>npm run verify:exposure</code>	Exposure aggregation
<code>npm run verify:scoring</code>	The Interchange Score
<code>npm run verify:onboarding</code>	Self-service member onboarding
<code>npm run filters:build</code>	Rebuild the membership filters
<code>npm run train</code>	Retrain the scoring models

Micro Eazy PWA — `cd pwa`

COMMAND	WHAT IT DOES
<code>npm run dev</code>	Vite dev server on :5173
<code>npm run build</code>	Production build to dist/
<code>npm run preview</code>	Serve dist/
<code>npm run lint</code>	ESLint

PBX Authentication — `cd pbx-auth`

A .NET background worker. No port, no browser surface, and a **third-party repository** — tracked read-only. Fork it before you need to push.

```
dotnet restore
dotnet run --project PBXAuthentication/PBXAuthentication.csproj
dotnet build -c Release
```

What changed today

Three things, and one of them will take the product down if it deploys in the wrong order.

RUN THIS BEFORE THE DEPLOY, NOT AFTER

A new column, `Org.systems`, backs the per-lender system switch. The deployed code selects it on the first screen every staff member sees. If the code ships before the column exists, `/suite` throws for everybody.

```
# against the PRODUCTION DATABASE_URL, before pushing to main
cd connected-suite
npx prisma generate
npm run db:push
```

1 — Lenders now hold systems, not the whole suite

There are now **three gates** in front of a system, and they compose by AND. Conflating any two of them is what makes access control impossible to explain to the person on the receiving end.

GATE	QUESTION	SET BY	WHERE
Entitled	Did this <i>organisation</i> buy it?	Platform admin	<code>/platform</code>
Visible	Is it part of <i>this person's</i> job?	Their administrator	<code>StaffUser.access</code>
Entered	Do they hold the right that opens it?	Their role	<code>Role.rights</code>

`/platform` now carries a colour-coded toggle row per lender. The change is staged rather than instant — ticking three boxes is one decision, not three — and the Save button names what is about to be *removed* before you press it, because removals are the half that hurts.

Null means every system. An organisation that existed before this column is unrestricted and inherits systems added later. An empty array is a different, real value meaning *no systems*, and the launcher says so plainly rather than rendering a blank page.

Hiding a tile is a courtesy; the control is the refusal. Every satellite layout now checks entitlement server-side, so a lender whose Ledgerly was switched off cannot reach it by typing `/books`.

2 — Seven front doors

Each system's bare host now serves *that system's own sign-in page* rather than dropping straight into the console. The page holds both audiences at once: somebody already carrying a BirgenAI ID gets one button with their own first name on it, and somebody who only has the link gets the real email-and-password form — on that system's artwork, in that system's colour.

HOST	SERVES
 lms.servicesuitecloud.com	/suite/lms/login
 analytics.servicesuitecloud.com	/suite/analytics/login
 peoplehub.servicesuitecloud.com	/suite/hr/login
 ledgerly.servicesuitecloud.com	/suite/accounting/login
 connectdesk.servicesuitecloud.com	/suite/callcenter/login
 microeazy.servicesuitecloud.com	/microeazy — the borrower app; /login is the staff door
 interchange.servicesuitecloud.com	Its own member gate, separate deployment

TWO HOSTS DELIBERATELY HAVE NO STAFF DOOR

microeazy.... keeps serving the borrower app at / because the installed PWA's start_url is that host — every home-screen icon already on a customer's phone opens there, and an installed base cannot be un-pointed. The lender staff door on that host is /login, which works today. The Interchange has no BirgenAI ID session to carry: it authenticates members by Ed25519 node certificate, so a "Continue as..." button would be a promise it cannot keep.

3 — The five satellites got a floor

The lending console has always sat on artwork while PeopleHub, Ledgerly, ConnectDesk and the Analytics Studio sat on flat grey — which is exactly why one of them looked like a product and the others looked like admin panels. They now share the console's canvas with a wash in their own accent, driven by the accent each system already declares. One shared image, seven distinct rooms, nothing new to keep in sync.

Separately: the lucide sparkles icon has been removed from every live surface and the rule recorded, so it does not come back.

Vercel: four projects

Three of them serve browsers. The fourth is a .NET worker and does not belong on Vercel at all.

PROJECT	REPOSITORY	ROOT DIRECTORY	FRAMEWORK	BUILD
connected-suite	BirgenAI_LMS	. (repo root)	Next.js	npm run build
interchange	Interchange	apps/interchange-console	Next.js	npm run build
micro-eazy-pwa	micro-eazy-pwa	. (repo root)	Vite	npm run build
pbx-auth	PBXAuthentication	Not a web app. Self-hosted Windows service — no Vercel project		

WHY THE INTERCHANGE RETURNS 404

The Interchange repository has **no package.json at its root**. The Next application is at apps/interchange-console. A Vercel project left on the default Root Directory therefore finds nothing to build, produces no error worth reading, and serves a 404.

The fix: Project Settings → General → Root Directory → apps/interchange-console → Save. Then redeploy.

AND THEN IT FAILS TO TYPE-CHECK — THE SECOND INTERCHANGE BLOCKER

With the Root Directory fixed, the build gets far enough to fail differently: Module '@prisma/client' has no exported member 'PrismaClient', followed by two dozen implicit-any errors that are all downstream of it.

@prisma/client/index.d.ts contains one line — export * from '.prisma/client/default' — a re-export of a directory that **prisma generate writes and npm install does not**. It exists on your machine and is gitignored, so Vercel installs from the lockfile and gets the empty stub. Nothing exports PrismaClient, every Prisma call loses its return type, and every .map(h => ...) becomes an implicit any under strict. One root cause, twenty errors.

The Connected Suite has always carried the generate step; the Interchange never did. Both now read:

```
"postinstall": "prisma generate",
"build":       "prisma generate && next build"
```

Both, not either. postinstall alone is skipped when Vercel restores a cached node_modules without re-running the install; build alone re-generates on every local build for no reason. Together they cover both paths, which is why this is the shape the Suite already used.

AND THEN IT FAILS A THIRD TIME — AT “COLLECTING PAGE DATA”

Failed to collect configuration for /api/authorise — [prisma] DATABASE_URL is not set.

That step *imports every route module* to read its configuration exports (runtime, dynamic, revalidate). Importing a route imported `lib/prisma.ts`, which built the client at module load and threw when the URL was absent. The client is now constructed lazily behind a Proxy, so importing the module costs nothing and the error fires at the first real query — where it is actually useful.

The general rule this is an instance of: **a build must not depend on a secret being in scope**. Build-time and run-time are different environments, and a build that only passes when production credentials happen to be present is one that fails on a laptop, in CI, and in every preview deployment.

REDEPLOY REBUILDS THE SAME COMMIT

All three of the above were fixed in successive commits, and each one appeared to “come back” because *Redeploy re-runs the commit that deployment was built from*, not the tip of the branch. If a fix looks like it did not take, check the commit SHA at the top of the build log against `git rev-parse --short origin/main` before debugging anything else. A fresh deploy comes from a push — `git commit --allow-empty -m "rebuild" && git push` if there is nothing to change.

Environment variables

■ connected-suite — the ones that make it a suite

VARIABLE	VALUE
SUITE_COOKIE_DOMAIN	.servicesuitecloud.com
SUITE_INTERCHANGE_ORIGIN	https://interchange.servicesuitecloud.com
PUBLIC_BASE_URL	https://microeazy.servicesuitecloud.com
NEXTAUTH_URL	https://microeazy.servicesuitecloud.com
DATABASE_URL	Supabase pooled — port 6543, ?pgbouncer=true
DIRECT_URL	Supabase direct — port 5432 (migrations)
NEXTAUTH_SECRET	Already set
VAULT_MASTER_KEY	Already set — rotating it orphans every stored credential

SUITE_COOKIE_DOMAIN IS WHAT MAKES THE SUITE A SUITE

A session cookie set on `lms.servicesuitecloud.com` with no explicit domain is *host-only* — the browser will simply not send it to `peoplehub.servicesuitecloud.com`. Single sign-on works in development purely because every system is the same host. Miss this one variable and the symptom is not an error: it is being asked to sign in again on every subdomain, which reads as six products rather than one.

Leave it *unset* on preview deployments. A cookie scoped to `.vercel1.app` would be shared with every other tenant of that domain.

Leave the six `SUITE*_ORIGIN` variables for the in-repo systems (`SUITE_LMS_ORIGIN`, `SUITE_HR_ORIGIN`, and so on) **unset**. They exist to split a system onto its own deployment later;

setting one now would point the launcher at an origin that does not exist yet.
SUITE_INTERCHANGE_ORIGIN is the exception, because that system genuinely is elsewhere.

■ interchange

VARIABLE	VALUE
DATABASE_URL	Its own Supabase project, pooled (:6543)
DIRECT_URL	Same project, direct (:5432)
INTERCHANGE_SESSION_SECRET	npm run keys:session — 32+ chars
INTERCHANGE_OPKF_KEY	Generate once. Rotating it re-tokenises every borrower in the ecosystem
CONSENT_WORDING_VERSION	2026-08-1.0
INTERCHANGE_ALLOW_CODE_LOGIN	0
INTERCHANGE_DEV_OPEN_CONSOLE	0

The last two matter. Four-digit operator codes are refused outright when NODE_ENV=production unless the first is explicitly 1 — the production door is the member certificate. Ten thousand combinations means the hash is not the control; lockout is.

■ micro-eazy-pwa

Whatever API base URL the client needs, and nothing else — it holds no lender data and no bureau data. vercel.json already rewrites every path to / for client-side routing.

Databases

You asked whether the Interchange should get a new Supabase database in production. It should — and not as a preference. It is the architecture's central claim.

SYSTEM	DATABASE	HOLDS
Connected Suite	Supabase project A (<i>existing</i>)	Lenders, borrowers, loans, staff, the vault
Interchange	Supabase project B (<i>new</i>)	Members, consent records, the audit trail. No borrower record, no raw identifier
PWA	None	Talks to the Suite over HTTP
PBX Auth	SQL Server	Telephony authentication

Why not one database with two schemas

Because the Registry is a separate *trust domain*, not merely a separate concern. It brokers between lenders who are competitors, and the thing being sold is that no member can read the exchange from their own systems. Share a database and a Suite credential becomes a Registry credential — the audit trail, the consent records and every member's query history all sit one connection string away from the application those members are being asked to trust.

There is a mundane failure too: `prisma db push` from either repository would see the other's tables, and Prisma's opinion about tables it does not know about is not one you want to discover in production.

Creating it

- 1 New Supabase project. Region **eu-central-1** or **eu-west-2** — whichever the Suite already uses, so a call between them does not cross an ocean.
- 2 Copy both connection strings from Project Settings → Database. The *Transaction* pooler (port 6543) is `DATABASE_URL`; append `?pgbouncer=true`. The *Session* / direct string (port 5432) is `DIRECT_URL`.
- 3 Push the schema and seed:

```
cd interchange/apps/interchange-console
# with the production URLs in the environment
npm run db:push
npm run db:seed
npm run operator -- create # the first super-admin
```

- 4 Set both URLs in the Vercel project, redeploy, then run `npm run verify:operator` against it — it asserts that a hand-written session cookie is refused.

TWO KEYS WITH VERY DIFFERENT BLAST RADII

`INTERCHANGE_SESSION_SECRET` can be rotated freely: it signs everyone out and nothing more. `INTERCHANGE_OPRF_KEY` cannot — rotating it re-tokenises every borrower in the ecosystem, which invalidates every consent reference and every exposure record that points at one. Generate it once, store it where you store the vault master key, and treat losing it as data loss.

DNS

Nine records at the registrar for `servicesuitecloud.com`. Seven of them point at the same Vercel project, which is the whole trick.

Step one — add the domains in Vercel first

Add every hostname under Project → Settings → Domains *before* touching the registrar. Vercel then shows you the exact target and issues the certificate the moment the record resolves. Adding the DNS first just means waiting through a propagation cycle for a certificate that had not been requested yet.

Step two — the records

	TYPE	NAME	VALUE	PROJECT
■	CNAME	microeazy	<code>cname.vercel-dns.com</code>	connected-suite
■	CNAME	connectedsuite	<code>cname.vercel-dns.com</code>	connected-suite
■	CNAME	lms	<code>cname.vercel-dns.com</code>	connected-suite
■	CNAME	analytics	<code>cname.vercel-dns.com</code>	connected-suite
■	CNAME	peoplehub	<code>cname.vercel-dns.com</code>	connected-suite
■	CNAME	ledgerly	<code>cname.vercel-dns.com</code>	connected-suite
■	CNAME	connectdesk	<code>cname.vercel-dns.com</code>	connected-suite
■	CNAME	interchange	<code>cname.vercel-dns.com</code>	interchange
■	CNAME	app	<code>cname.vercel-dns.com</code>	micro-eazy-pwa

If you want the apex `servicesuitecloud.com` itself to resolve, add an A record for `@` pointing at `76.76.21.21`. Vercel will confirm the current value when you add the domain — use what it tells you rather than what is written here, because that address is the one thing on this page that can change without anybody telling us.

SET MICROEAZY AS THE PRODUCTION DOMAIN

In the connected-suite project, mark `microeazy.servicesuitecloud.com` as the production domain. Every other host on that project should be added as a plain domain, *not* a redirect — a redirect would collapse all seven front doors into one and undo the entire point of `src/proxy.ts`.

Step three — prove it

```
# each of these should return 200 and land on a different door
curl -sI https://lms.servicesuitecloud.com | head -1
curl -sI https://connectdesk.servicesuitecloud.com | head -1
curl -sI https://interchange.servicesuitecloud.com | head -1

# the staff door on the canonical host
curl -sI https://microeazy.servicesuitecloud.com/login | head -1

# the cookie must carry the parent domain, or SSO does not cross
curl -sI https://microeazy.servicesuitecloud.com/api/auth/login \
| grep -i set-cookie # expect: Domain=.servicesuitecloud.com
```


Go-live, in order

The order matters in exactly two places: the migration must precede the deploy, and the Vercel domains must precede the DNS. Everything else is independent.

- 1 **Migrate the Suite database.** The new `Org.systems` column, against production, before any code that reads it exists.

```
cd connected-suite && npm run db:push
```

- 2 **Push the Suite.** Vercel builds on push to `main`. This is the deploy that lights up `connectedsuite.servicesuitecloud.com` and the six doors.

```
cd connected-suite
git add -A && git commit -m "Seven doors, per-lender systems"
git push origin main
```

- 3 **Create the Interchange database** and push its schema — see 06.

- 4 **Fix the Interchange project's Root Directory** to `apps/interchange-console`, set its environment variables, and redeploy. The 404 goes away on that redeploy and not before.

- 5 **Push the Interchange and the PWA.**

```
cd interchange && git push origin main
cd ../pwa && git push origin main
```

- 6 **Add all nine domains in Vercel**, then the nine DNS records at the registrar. Wait for the certificates to issue — usually minutes.

- 7 **Set `SUITE_COOKIE_DOMAIN`** to `.servicesuitecloud.com` and `SUITE_INTERCHANGE_ORIGIN` to the live Interchange, then redeploy the Suite. Do this *after* the domains resolve: a cookie scoped to a domain that does not serve the app yet signs everybody out.

- 8 **Bump the pins.** Record which combination of commits is now live.

```
cd "C:\GIT\MICRO EAZY"
git add connected-suite interchange pwa
git commit -m "Live on servicesuitecloud.com" && git push
```

- 9 **Walk it.** Sign in at `microeasy.servicesuitecloud.com/login`, land on the launcher, open each system through its own door, and switch between two of them using the top-right menu without being asked for a password. Then switch a system off at `/platform` and reload the launcher — the tile should be gone, and typing its route directly should bounce you back.

VERIFIED TODAY

`npx tsc --noEmit` clean and `npm run build` exit 0 on the Connected Suite with all of the above in place; `tsc` clean on the Interchange console. `npm run art:check` runs and reports 4 plates passing, 2 failing. Nothing here has been run against production.

Still open

What is not done. None of it blocks the deploy; all of it is visible to somebody.

Two login plates are wrong, and one of them measurably so

`npm run art:check` scores every plate on three things the eye cannot judge while looking at the plate itself: whether the left third is dark enough for the sign-in card, whether there is any chroma at all, and whether the hue matches the system's accent.

	PLATE	PIXELS	CHROMA	HUE ERROR	
	login-lending	1586×992	28.0%	2°	ok
	login-portal	1536×944	17.3%	8°	ok
	login-analytics	1586×992	26.2%	7°	ok
	login-desk	1536×944	7.1%	20°	thin — redo next
	login-people	1344×768	1.1%	12°	regenerate
	login-books	1536×944	2.8%	140°	regenerate

PeopleHub is a black rectangle with four flat violet bars — the generator took *"most dark, some glowing"* literally — and it is the lowest-resolution file in the set. **Ledgerly** is a beautiful photograph of the wrong colour: 35° gold against its 175° green-teal, because *"antique brass"* is a colour instruction that outweighs a rim-light qualifier eleven words later. Both prompts in `src/lib/suite/artwork.ts` have been rewritten for those specific failures.

Four assets worth buying, none of them login plates

FILE	WHY
<code>images/white-background.png</code>	Highest value. Currently 740×465 and used behind nine surfaces — now including all five satellites. It is upscaled 3.5× on a normal monitor. Regenerate at 2560×1600, same filename, no code change
<code>images/dark-background.png</code>	The launcher builds its dark field from seven CSS blur circles. One plate is cheaper and richer
<code>interchange → public/</code>	Still holds the Next.js starter SVGs. Needs an <code>og.png</code> (1200×630), <code>icon-512.png</code> , <code>apple-touch-icon.png</code>
<code>public/og/*.png</code>	Every WhatsApp and Slack link to any of these systems previews as blank. 1200×630 per system, croppable from the login masters

Prompts and target paths for all of it are in `connected-suite/public/images/suite/README.md`.

Not built

- **System selection at organisation creation.** The toggles exist at `/platform` for an org that already exists. The self-service onboarding flow at `/onboard` does not yet ask, so a new lender starts unrestricted and is narrowed afterwards.
- **Package and systems are independent.** Deliberately — a plan change should never silently take a system away — but it does mean the two can disagree and nothing flags it.
- **The Interchange is not in the switcher's session.** Its tile links out and opens its own gate. Federating a BirgenAI ID into an Ed25519 member session is a real piece of work, not a configuration change.

- **PBX Authentication is a third-party repository** (`skegode`), tracked read-only. Fork it before you need to push.

Where to look when this document is wrong. `ecosystem/registry.json` declares every system, port, command and deployment target, and every orchestration script reads it. `ecosystem/ARCHITECTURE.md` holds the boundaries that are load-bearing. `src/lib/suite/` holds the seven systems, their hosts, their artwork and their entitlements — one list each, no second place. Adding an eighth system is one registry entry, one `SUITE_APPS` entry and one `git submodule add`.